



Advanced Uses for Lists

Part 1: Multidimensional Lists

Bringing lists into the 2nd
Dimension... and BEYOND!

Multidimensional Lists

A **multidimensional list** is a list that contains other lists.

These nested lists can be used to represent structures such as tables, grids, layers, or grouped data.

Each additional level of nesting adds another dimension to the structure.

2D Lists

A **2D list** is a list containing other lists, commonly representing a table or grid.

The outer list contains rows, while each inner list contains the values in that row.

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

Creating a 2D List

A 2D list can be created manually by placing several lists inside another list.

Each inner list usually represents one row of related data.

Keeping each row the same length makes the structure easier to work with.

```
students = [  
    ["Alex", 82, 91],  
    ["Sam", 75, 88],  
    ["Jordan", 94, 90]  
]
```

Rows and Columns in 2D Lists

In a 2D list, each inner list usually represents a row, while values in the same position form columns.

Rows often store one complete record, while columns represent the same type of information across different records.

This makes 2D lists useful for keeping related data organized.

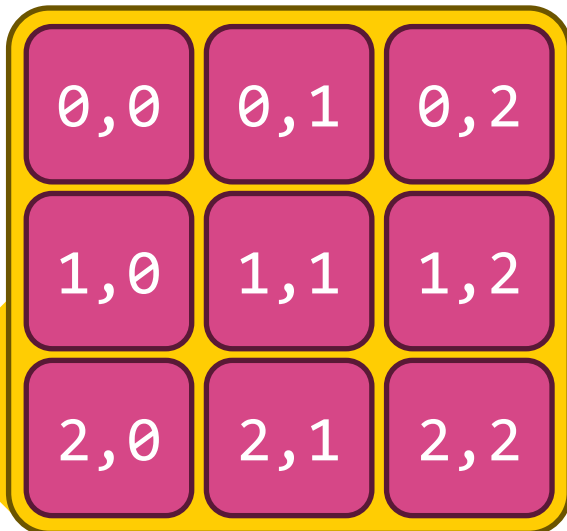
```
students = [  
    # Name ,      Grade, Score  
    ["Alex",      10,    91],  
    ["Sam",       11,    88],  
    ["Jordan",    10,    90]  
]
```

Accessing Values in 2D Lists

Values in a 2D list are accessed using two indices.

The first index selects the inner list, and the second index selects the value inside it.

This is commonly written as `list[row][column]`.



0,0	0,1	0,2
1,0	1,1	1,2
2,0	2,1	2,2

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(grid[1][2])
```

OUTPUT

6

Traversing with Nested Loops

A 2D list can be traversed using one loop for the rows and another loop for the values inside each row.

This works well when you need each value but do not need its position.

The inner loop completes before the outer loop moves to the next row.

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6]  
]  
  
for row in grid:  
    for value in row:  
        print(value)
```

Traversing with Indices in Nested Loops

Nested loops can also use indices when the position of each value matters.

One loop tracks the row index and another tracks the column index.

This gives direct access to each value using `list[i][j]`.

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6]  
]  
  
for i in range(len(grid)):  
    for j in range(len(grid[i])):  
        print(grid[i][j])
```

Row Major Traversal

Row major traversal processes every value in one row before moving to the next row.

This is the most natural way to traverse a typical 2D list in Python.

The outer loop handles rows, while the inner loop handles columns.

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6]  
]  
  
for i in range(len(grid)):  
    for j in range(len(grid[i])):  
        print(grid[i][j])
```

Column Major Traversal

Column major traversal processes values down each column before moving to the next column.

This usually requires working with indices because Python stores the structure by rows.

The column loop runs on the outside while the row loop runs inside.

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6]  
]  
  
for j in range(len(grid[0])):  
    for i in range(len(grid)):  
        print(grid[i][j])
```

3D Lists and Beyond

A 3D list contains 2D lists inside another list.

It can represent structures such as multiple grids, layers, or groups of tables.

Python does not place a hard limit on how deeply lists can be nested, although deeply nested structures become harder to manage.

```
levels = [  
    [  
        [1, 2],  
        [3, 4]  
    ],  
    [  
        [5, 6],  
        [7, 8]  
    ]  
]
```

Accessing Values in 3D and Beyond Lists

Each additional dimension requires another index.

A 3D list therefore uses three indices, such as `list[i][j][k]`.

The same pattern continues for deeper nested lists.

```
levels = [  
    [  
        [1, 2],  
        [3, 4]  
    ],  
    [  
        [5, 6],  
        [7, 8]  
    ]  
]  
print(levels[1][0][1])
```

Traversing 3D and Beyond Lists

Traversing deeper lists requires another nested loop for each dimension.

A 3D list usually requires three loops, while a 4D list may require four.

As dimensions are added, the number of values that may need to be processed can grow very quickly.

```
levels = [  
    [  
        [1, 2],  
        [3, 4]  
    ],  
    [  
        [5, 6],  
        [7, 8]  
    ]  
]  
  
for level in levels:  
    for row in level:  
        for value in row:  
            print(value)
```